APPENDIX A

Complete R Code for Statistical Analysis — Chapters 4 and 5

The R code presented in this appendix covers all statistical analyses reported in Chapters 4 and 5 of this dissertation. The code is organised in eleven sections corresponding to the analytical sequence followed in those chapters. All code was executed in R version 4.3.1 using RStudio. The five input CSV files — yearly_ndvi_2001_2024.csv, monthly_ndvi_2001_2024.csv, yearly_zonal_stats_2001_2024.csv, monthly_zonal_stats_2001_2024.csv, and validation_2025_monthly_ndvi.csv — are available as part of the dissertation data package and were exported from Google Earth Engine as described in Chapter 3 and Appendix A of that chapter.

A.1 Data Import, Preprocessing, and Descriptive Statistics

This section contains the R code for importing the yearly NDVI dataset, selecting and renaming the required variables, performing missing value assessment, computing descriptive statistics, and producing the three-year moving average. These operations correspond to Sections 4.2.1 and 4.2.2 of Chapter 4.

A.1.1 Package Loading and Data Import

```
# ── PACKAGE LOADING ──────────────────────────────────────────
library(readr)      # Data import
library(dplyr)      # Data manipulation
library(ggplot2)    # Visualisation
library(forecast)   # ARIMA and time series modelling
library(tseries)    # Stationarity tests (ADF, KPSS)
library(trend)      # Mann-Kendall test and Sen's Slope
library(lmtest)     # Breusch-Pagan test
library(car)        # Durbin-Watson test
library(rstanarm)   # Bayesian regression (Chapter 5)
library(bayesplot) # Bayesian diagnostic plots (Chapter 5)

# ── YEARLY DATA IMPORT ───────────────────────────────────────
```

```
ndvi_data <- read_csv('yearly_ndvi_2001_2024.csv')

ndvi_data <- ndvi_data %>%
  select(year, mean_ndvi) %>%
  rename(Year = year, NDVI = mean_ndvi)

str(ndvi_data)           # Confirm numeric format
colSums(is.na(ndvi_data)) # Confirm no missing values
```

A.1.2 Descriptive Statistics

```
summary(ndvi_data$NDVI)                              # Five-number summary
sd(ndvi_data$NDVI)                                   # Standard deviation
var(ndvi_data$NDVI)                                  # Variance
IQR(ndvi_data$NDVI)                                  # Interquartile range
cv <- sd(ndvi_data$NDVI)/mean(ndvi_data$NDVI)*100
round(cv, 2)                                         # Coefficient of
variation
```

A.1.3 Three-Year Moving Average

```
ndvi_data$MA3 <- stats::filter(ndvi_data$NDVI,
                               filter = rep(1/3, 3),
                               sides  = 2)
print(ndvi_data, n = 24)  # View complete dataset with moving average
```

A.2 Linear Regression, Diagnostic Tests, and Non-Parametric Trend Analysis

This section provides the R code for fitting the OLS linear regression model, running all diagnostic tests, and applying the Mann-Kendall test with Sen's Slope estimator. These correspond to Sections 4.2.3 through 4.2.5 of Chapter 4.

A.2.1 OLS Linear Regression (Figure 4.1)

```
# ── FIT OLS MODEL ──────────────────────────────────────────────
trend_model <- lm(NDVI ~ Year, data = ndvi_data)
summary(trend_model)                # Full summary
coef(trend_model)                   # Intercept and slope
confint(trend_model)                # 95% confidence intervals
summary(trend_model)$r.squared      # R-squared = 0.8676
summary(trend_model)$adj.r.squared  # Adjusted R-squared

# ── FIGURE 4.1: ANNUAL NDVI TREND PLOT ─────────────────────────
fig_4_1 <- ggplot(ndvi_data, aes(x = Year, y = NDVI)) +
  geom_line(colour = '#1a7a4a', linewidth = 1.0) +
  geom_point(colour = '#1a7a4a', size = 2.5) +
  geom_smooth(method = 'lm', colour = 'red',
              linetype = 'dashed', se = FALSE, linewidth = 1.0) +
```

```
  geom_line(aes(y = MA3), colour = '#2e75b6',
            linewidth = 0.8, na.rm = TRUE) +
  labs(title = 'Annual Mean NDVI — Aligarh District (2001–2024)',
       subtitle = 'OLS slope = 0.00335 per year | R² = 0.868 | p <
0.001',
       x = 'Year', y = 'Mean NDVI') +
  theme_minimal(base_size = 12)
print(fig_4_1)
```

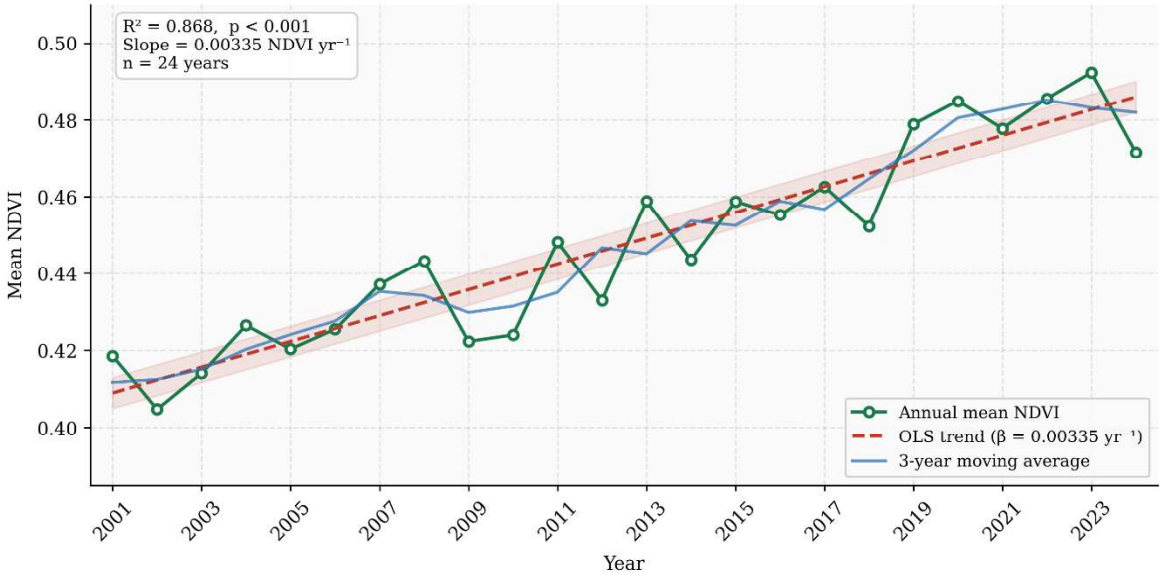


Figure 4.1: Temporal Trend in Annual Mean NDVI for Aligarh District (2001–2024). Dark green line = observed NDVI values; red dashed line = OLS regression fit (slope = 0.00335 yr^{-1}, $R^2 = 0.868$, $p < 0.001$); blue line = 3-year moving average.

A.2.2 Regression Diagnostics (Figure 4.2)

```
# ── FOUR-PANEL DIAGNOSTIC PLOT ──────────────────────────────────
par(mfrow = c(2, 2))
plot(trend_model)       # Figure 4.2
par(mfrow = c(1, 1))

# ── DURBIN-WATSON TEST (residual autocorrelation) ──────────────
dwtest(trend_model)     # H0: no autocorrelation | p = 0.665 → retained

# ── BREUSCH-PAGAN TEST (heteroscedasticity) ────────────────────
bptest(trend_model)     # H0: homoscedasticity   | p = 0.322 → retained

# ── COOK'S DISTANCE (influential observations) ─────────────────
cooks_d   <- cooks.distance(trend_model)
threshold <- 4 / nrow(ndvi_data)  # Standard threshold = 4/n
which(cooks_d > threshold)          # Identify observations above
threshold
```

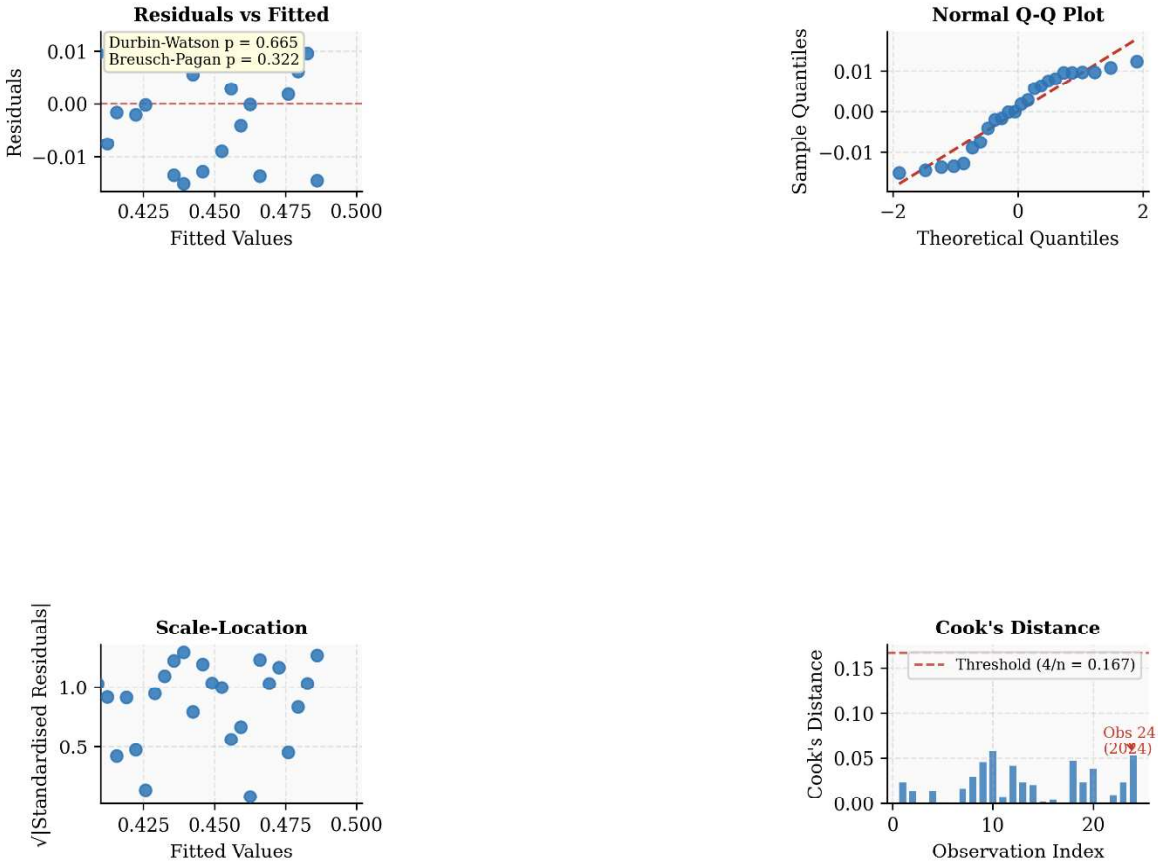


Figure 4.2: Regression Diagnostic Plots for the Yearly NDVI Linear Trend Model. Durbin-Watson p = 0.665; Breusch-Pagan p = 0.322. No serious violation of regression assumptions detected.

A.2.3 Mann-Kendall Test and Sen's Slope (Figure 4.3)

```
# ── MANN-KENDALL TEST ──────────────────────────────────────────
mk_result <- mk.test(ndvi_data$NDVI)
print(mk_result)
mk_result$statistic   # Z = 5.2833
mk_result$estimates   # Kendall's tau = 0.775
mk_result$p.value     # p = 1.27e-07

# ── SEN'S SLOPE ──────────────────────────────────────────────────
sens_result <- sens.slope(ndvi_data$NDVI)
print(sens_result)
sens_result$estimates  # Slope = 0.00340 NDVI yr⁻¹
sens_result$conf.int   # 95% CI: [0.00277, 0.00409]

# ── FIGURE 4.3: SEN'S SLOPE PLOT ─────────────────────────────────
sen_df <- data.frame(Year = ndvi_data$Year, NDVI = ndvi_data$NDVI)
sen_slope_val <- as.numeric(sens_result$estimates)
sen_intercept <- median(ndvi_data$NDVI) -
                 sen_slope_val * median(ndvi_data$Year)
fig_4_3 <- ggplot(sen_df, aes(x = Year, y = NDVI)) +
```

```
  geom_point(colour = '#1a7a4a', size = 2.5) +
  geom_abline(slope = sen_slope_val,
              intercept = sen_intercept,
              colour = 'darkorange', linewidth = 1.1) +
  labs(title = "Sen's Slope Trend Estimation — Aligarh District (2001–
2024)",
       subtitle = paste('Slope =', round(sen_slope_val,5),
                        '| 95% CI: [0.00277, 0.00409]',
                        '| Mann-Kendall Z = 5.28, p < 0.001'),
       x = 'Year', y = 'Mean NDVI') +
  theme_minimal(base_size = 12)
print(fig_4_3)
```

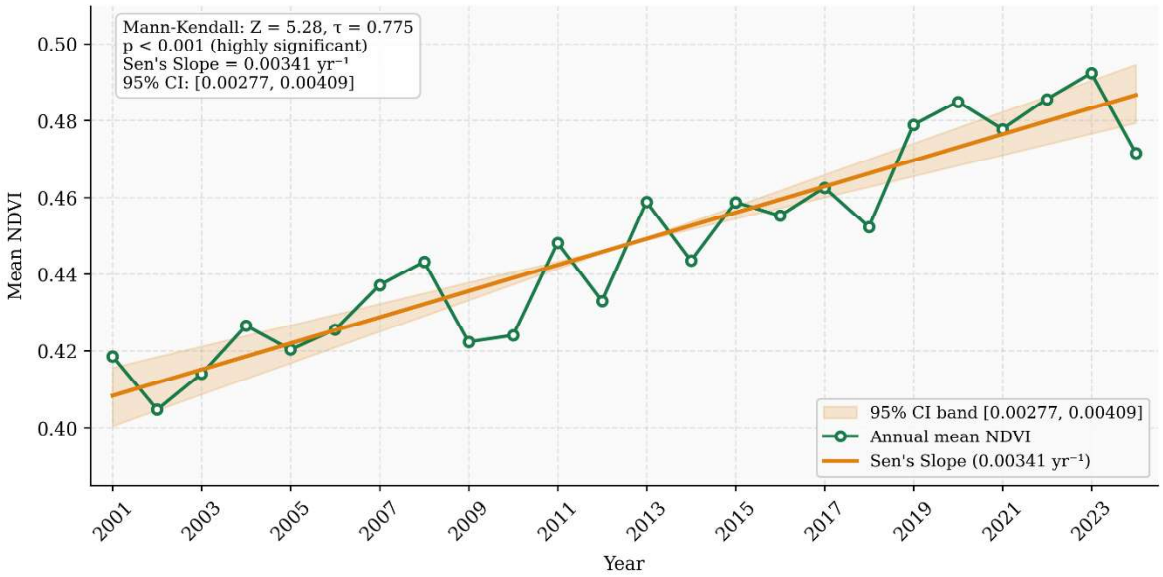


Figure 4.3: Sen's Slope Non-Parametric Trend Estimation for Aligarh District (2001–2024). Orange line = median slope estimate (0.00340 yr⁻¹); shaded band = 95% confidence interval [0.00277, 0.00409]. Mann-Kendall Z = 5.28, τ = 0.775, p < 0.001.

A.3 Time Series Conversion and Stationarity Testing

This section contains the R code for converting the yearly NDVI series to a time series object and performing stationarity testing using ADF, PP, and KPSS tests. These operations correspond to Sections 4.2.6 and 4.2.7 of Chapter 4.

```
# ── TIME SERIES OBJECT ──────────────────────────────────────────
ndvi_ts <- ts(ndvi_data$NDVI, start = 2001, frequency = 1)
print(ndvi_ts); class(ndvi_ts)
```

```
# ── STATIONARITY TESTS — ORIGINAL SERIES ──────────────────────────
adf_result  <- adf.test(ndvi_ts, alternative = 'stationary')
pp_result   <- pp.test(ndvi_ts)
kpss_result <- kpss.test(ndvi_ts, null = 'Level')
print(adf_result)   # p = 0.3561 — Non-stationary
print(pp_result)    # p = 0.0100 — Conflicting result
print(kpss_result)  # p = 0.0100 — Non-stationary (null rejected)

# ── FIRST DIFFERENCING ──────────────────────────────────────
ndvi_diff <- diff(ndvi_ts, differences = 1)

# ── STATIONARITY TESTS — DIFFERENCED SERIES ──────────────────────
adf.test(ndvi_diff, alternative = 'stationary')  # p = 0.024
pp.test(ndvi_diff)                               # p < 0.01
kpss.test(ndvi_diff, null = 'Level')             # p = 0.10
# All three confirm stationarity after first differencing. d = 1
```

A.4 ACF/PACF Analysis and ARIMA Model Fitting

This section contains the R code for ACF and PACF analysis, ARIMA model fitting and comparison, residual diagnostics, and five-year forecasting. These operations correspond to Sections 4.2.8 through 4.2.11 of Chapter 4.

A.4.1 ACF and PACF Plots (Figure 4.4)

```
# ── ACF AND PACF OF DIFFERENCED SERIES ──────────────────────────
par(mfrow = c(1, 2))
acf(ndvi_diff,  lag.max = 12,
    main = 'ACF — First-Differenced Yearly NDVI')
pacf(ndvi_diff, lag.max = 12,
    main = 'PACF — First-Differenced Yearly NDVI')
par(mfrow = c(1, 1))
# Significant spike at lag 1 in both ACF and PACF -> ARIMA(1,1,0)
```

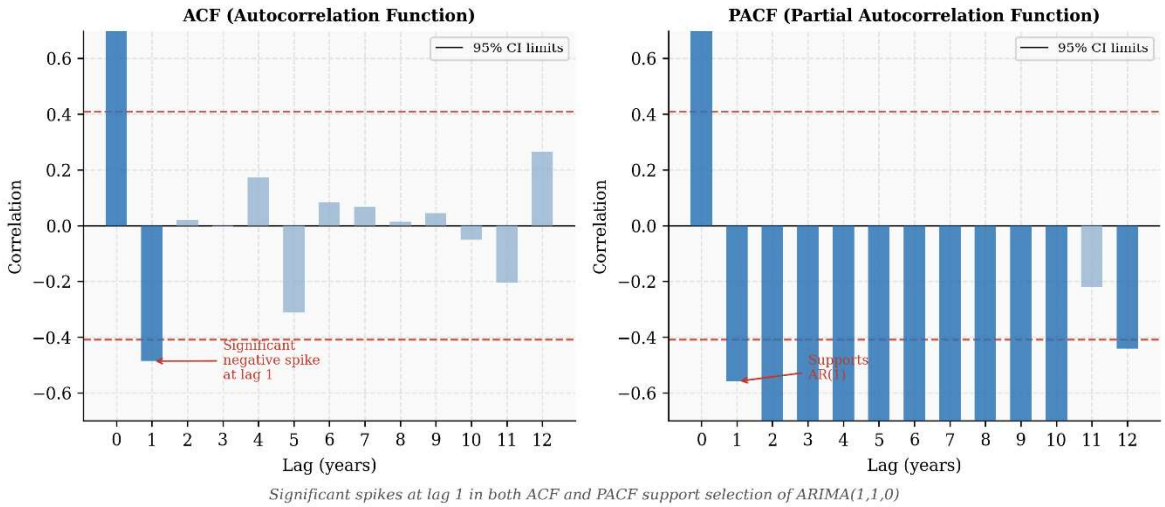


Figure 4.4: ACF and PACF of First-Differenced Yearly NDVI Series. Dashed lines = 95% confidence limits (±1.96/√n). Significant spike at lag 1 supports ARIMA(1,1,0) selection.

A.4.2 ARIMA Model Comparison and Selection

```
model_110 <- Arima(ndvi_ts, order = c(1, 1, 0))
model_011 <- Arima(ndvi_ts, order = c(0, 1, 1))
model_111 <- Arima(ndvi_ts, order = c(1, 1, 1))

AIC(model_110, model_011, model_111)  # ARIMA(1,1,0): AIC = -101.28
BIC(model_110, model_011, model_111)  # ARIMA(1,1,0): BIC = -99.29
accuracy(model_110)  # RMSE = 0.00895 — selected model
summary(model_110)
```

A.4.3 ARIMA Residual Diagnostics

```
checkresiduals(model_110)             # Comprehensive diagnostic plot
Box.test(residuals(model_110),
         lag = 12, type = 'Ljung-Box') # p = 0.158 -> White noise
acf(residuals(model_110), main = 'ACF of Residuals')
hist(residuals(model_110), main = 'Residual Histogram',
     xlab = 'Residuals', col = 'steelblue', border = 'white')
qqnorm(residuals(model_110)); qqline(residuals(model_110))
```

A.4.4 Five-Year NDVI Forecast 2025–2029 (Figure 4.5)

```
fc_yearly <- forecast(model_110, h = 5, level = 95)
print(fc_yearly)       # Point forecasts and prediction intervals
as.data.frame(fc_yearly) # Extract mean, Lo 95, Hi 95 columns

# ── FIGURE 4.5: FORECAST PLOT ──────────────────────────────────
fig_4_5 <- autoplot(fc_yearly) +
  labs(title = 'Annual NDVI Forecast — Aligarh District (2025–2029)',
       subtitle = 'Model: ARIMA(1,1,0) | 95% Prediction Interval',
       x = 'Year', y = 'Mean NDVI') +
  theme_minimal(base_size = 12)
print(fig_4_5)
```

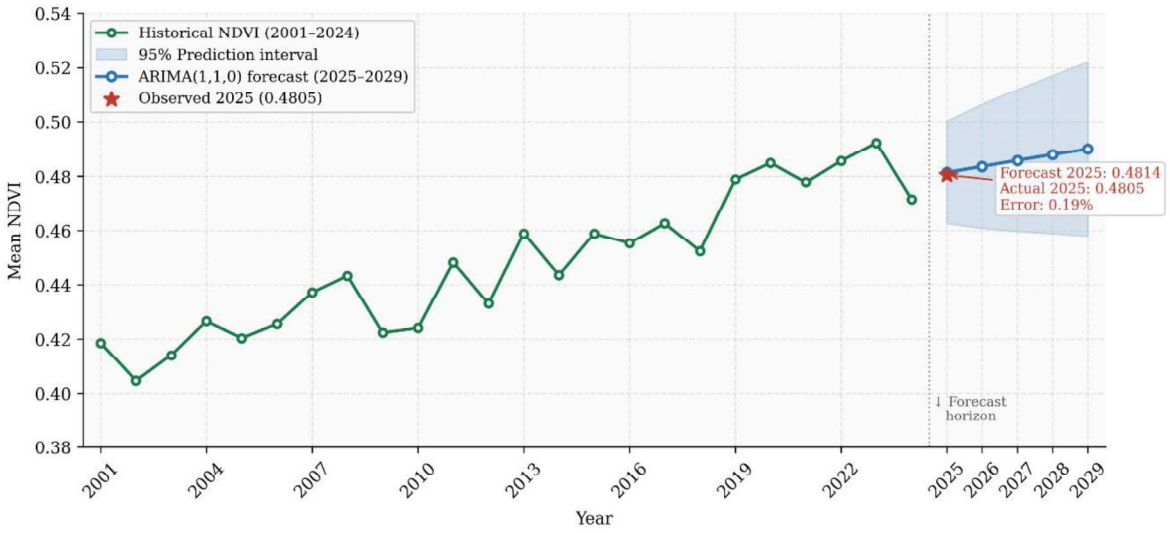


Figure 4.5: Projected Annual Mean NDVI for Aligarh District (2025–2029). ARIMA(1,1,0) model. Blue line = point forecast; shaded band = 95% prediction interval. Red point = observed 2025 value (0.4805). Forecast error = 0.19%.

A.5 Yearly Forecast Validation

This section provides the R code for validating the ARIMA(1,1,0) model forecast against the observed 2025 annual NDVI. The validation corresponds to Section 4.2.12 of Chapter 4.

```
# ── IMPORT VALIDATION DATA ──────────────────────────────────────
validation_2025 <- read_csv('validation_2025_monthly_ndvi.csv')

# ── COMPUTE OBSERVED 2025 ANNUAL MEAN ──────────────────────────
observed_2025   <- mean(validation_2025$mean_ndvi)
round(observed_2025, 4)   # = 0.4805

# ── EXTRACT FORECASTED 2025 VALUE ──────────────────────────────
forecasted_2025 <- as.numeric(fc_yearly$mean[1])
round(forecasted_2025, 4) # = 0.4814

# ── VALIDATION METRICS
──────────────────────────────────────────
abs_error <- abs(observed_2025 - forecasted_2025)
pct_error <- (abs_error / observed_2025) * 100
cat('Absolute Error:', round(abs_error, 4), '\n')  # 0.0009
cat('Percentage Error:', round(pct_error, 2), '%\n') # 0.19%

# ── FIGURE 4.6: VALIDATION BAR CHART ───────────────────────────
val_df <- data.frame(
  Type = c('Forecasted (2025)', 'Observed (2025)'),
```

```
  NDVI = c(forecasted_2025, observed_2025))
fig_4_6 <- ggplot(val_df, aes(x = Type, y = NDVI, fill = Type)) +
  geom_bar(stat = 'identity', width = 0.45, colour = 'white') +
  geom_text(aes(label = round(NDVI, 4)),
            vjust = -0.4, size = 4.5, fontface = 'bold') +
  scale_fill_manual(values = c('#2e75b6', '#1a7a4a')) +
  labs(title = 'Yearly NDVI Forecast Validation — 2025',
       subtitle = 'Absolute Error = 0.0009 | Percentage Error =
0.19%',
       x = NULL, y = 'Mean NDVI') +
  theme_minimal(base_size = 12) + theme(legend.position = 'none')
print(fig_4_6)
```

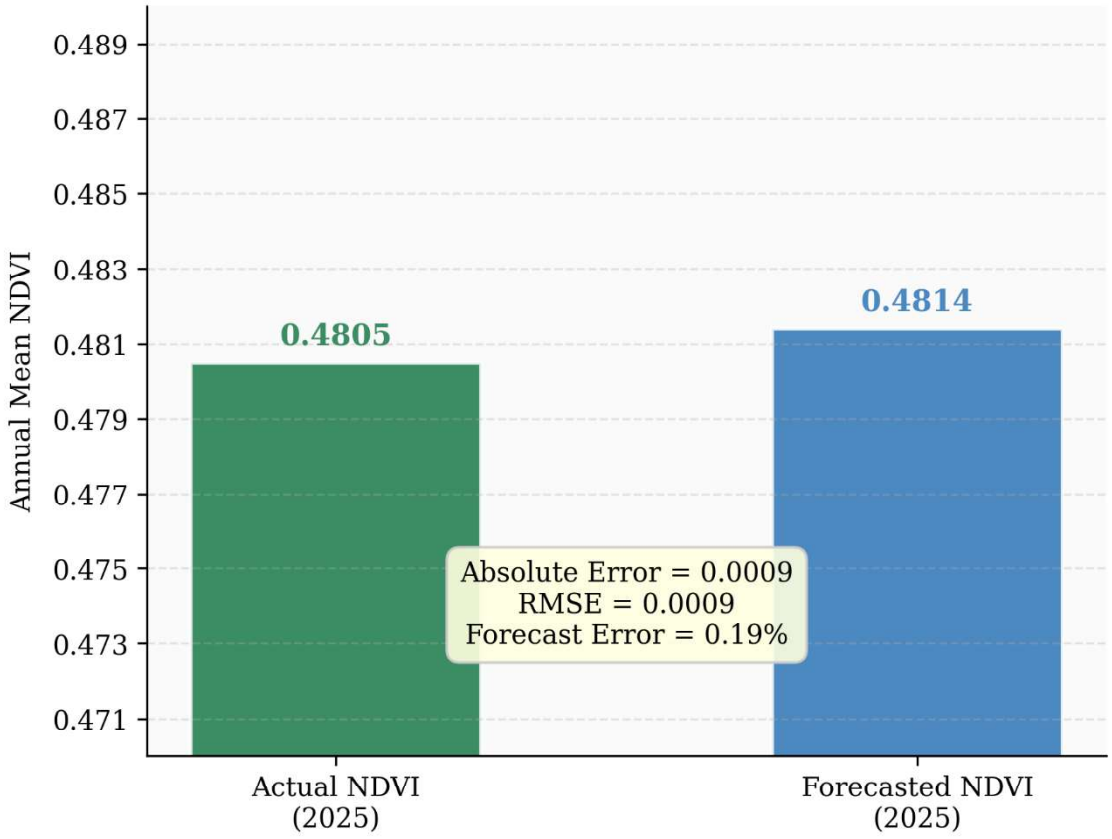


Figure 4.6: Yearly NDVI Forecast Validation — Aligarh District (2025). Absolute error = 0.0009; Percentage error = 0.19%.

A.6 Monthly NDVI Analysis — Data Import, Descriptive Statistics, and Visualisation

This section contains the R code for importing and preprocessing the monthly NDVI dataset, computing descriptive statistics, generating the monthly time series plot, the seasonal

average bar chart, and performing STL decomposition. These correspond to Sections 4.3.1 through 4.3.5 of Chapter 4.

A.6.1 Monthly Data Import

```
monthly_data <- read_csv('monthly_ndvi_2001_2024.csv')
monthly_data <- monthly_data %>%
  select(year, month, mean_ndvi) %>%
  rename(Year = year, Month = month, NDVI = mean_ndvi) %>%
  arrange(Year, Month)
monthly_data$Date <- as.Date(
  paste(monthly_data$Year, monthly_data$Month, '01', sep='-'))
dim(monthly_data)               # 288 x 4
colSums(is.na(monthly_data))    # Confirm no missing values
monthly_ts <- ts(monthly_data$NDVI, start=c(2001,1), frequency=12)
```

A.6.2 Monthly Descriptive Statistics

```
summary(monthly_ts)
sd(monthly_data$NDVI)    # = 0.1383
var(monthly_data$NDVI)   # = 0.0191
monthly_avg <- monthly_data %>%
  group_by(Month) %>%
  summarise(Avg_NDVI = round(mean(NDVI),4),
            SD_NDVI  = round(sd(NDVI),4))
print(monthly_avg, n=12)
```

A.6.3 Monthly Time Series Plot (Figure 4.7)

```
fig_4_7 <- ggplot(monthly_data, aes(x = Date, y = NDVI)) +
  geom_line(colour = '#1a7a4a', linewidth = 0.5) +
  labs(title = 'Monthly NDVI Time Series — Aligarh District (2001–
2024)',
       x = 'Date', y = 'Mean NDVI') +
  theme_minimal(base_size = 11)
print(fig_4_7)
```

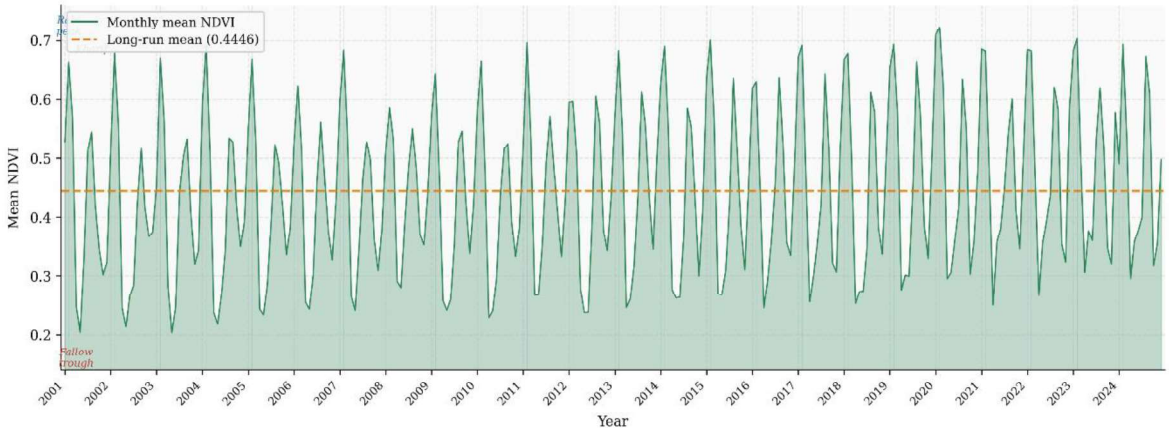


Figure 4.7: Monthly NDVI Time Series for Aligarh District (January 2001 to December 2024). n = 288 observations. Seasonal oscillation reflects the Rabi–Kharif agricultural cycle.

A.6.4 Monthly Seasonal Average Plot (Figure 4.8)

```
month_names <- c('Jan','Feb','Mar','Apr','May','Jun',
                 'Jul','Aug','Sep','Oct','Nov','Dec')
monthly_avg$Month_Name <- factor(month_names,levels=month_names)

fig_4_8 <- ggplot(monthly_avg,
    aes(x = Month_Name, y = Avg_NDVI, fill = Avg_NDVI)) +
  geom_bar(stat = 'identity', colour = 'white') +
  geom_errorbar(aes(ymin = Avg_NDVI - SD_NDVI,
                    ymax = Avg_NDVI + SD_NDVI),
                width = 0.3, colour = 'grey30') +
  scale_fill_gradient(low='#ffffcc', high='#238443',
                      name='Mean NDVI') +
  labs(title = 'Monthly Seasonal Average NDVI — Aligarh District
(2001–2024)',
       subtitle = 'Error bars = ±1 standard deviation across 24
years',
       x = 'Month', y = 'Average NDVI') +
  theme_minimal(base_size = 11)
print(fig_4_8)
```

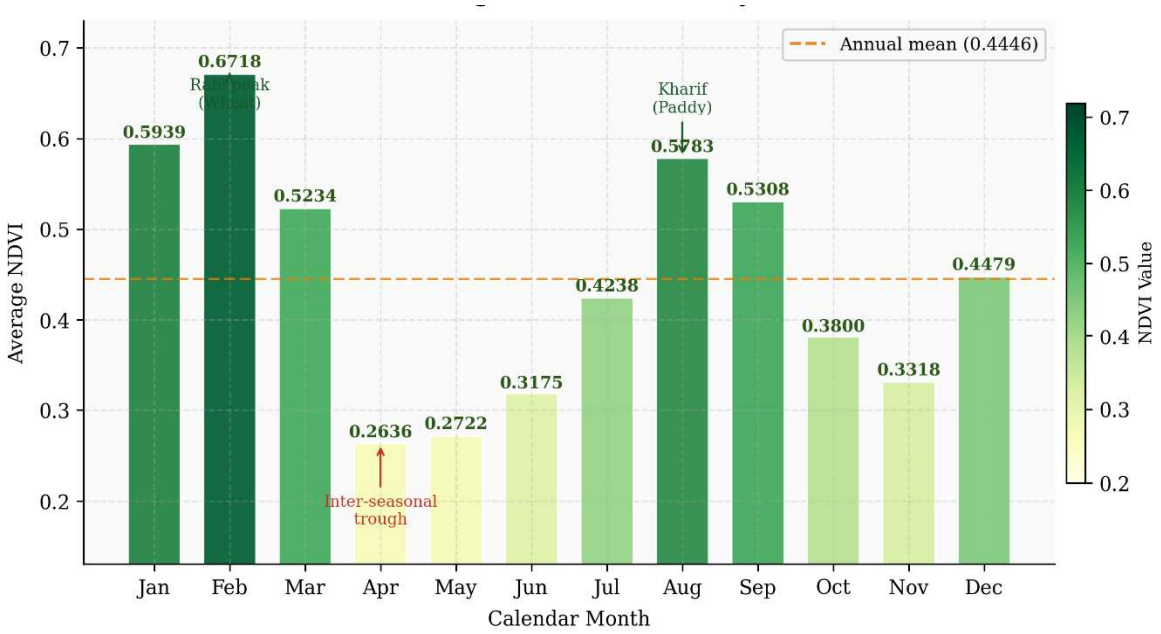


Figure 4.8: Monthly Seasonal Average NDVI for Aligarh District (2001–2024). Bar height = 24-year monthly mean; error bars = ±1 standard deviation. February peak (0.6718) corresponds to peak Rabi crop development. April–May trough (0.26–0.27) reflects the inter-seasonal fallow period.

A.6.5 STL Decomposition (Figure 4.9)

```
stl_result <- stl(monthly_ts,
                  s.window = 'periodic',
                  t.window = 13,
                  robust   = TRUE)
```

```
plot(stl_result,
     main = 'STL Decomposition of Monthly NDVI — Aligarh District')

trend_comp    <- stl_result$time.series[,'trend']
seasonal_comp <- stl_result$time.series[,'seasonal']
remainder_comp<- stl_result$time.series[,'remainder']

# Seasonal strength (Wang et al., 2006)
Fs <- max(0, 1 -
var(remainder_comp)/var(seasonal_comp+remainder_comp))
round(Fs, 3)    # = 0.941 — Very strong seasonality confirmed
```

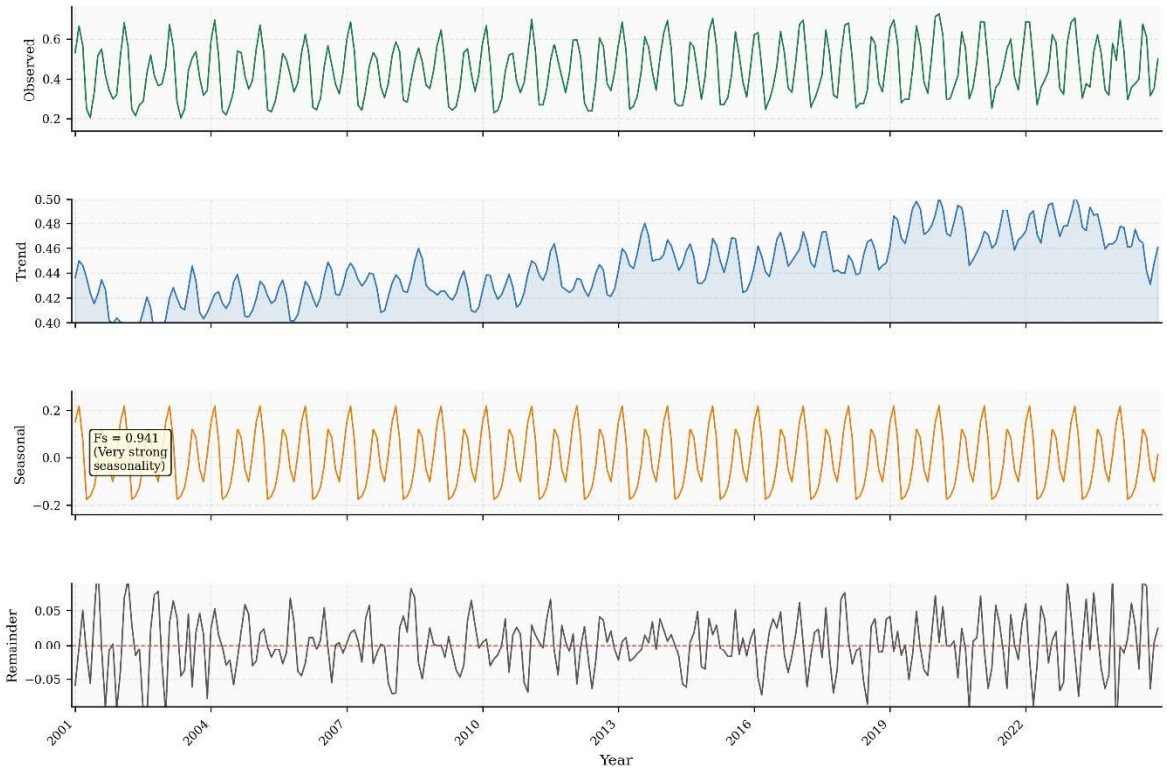


Figure 4.9: STL Decomposition of Monthly NDVI for Aligarh District (2001–2024). Four panels: observed series, seasonal component, trend component, remainder. Seasonal strength Fs = 0.941.

A.7 Monthly Stationarity Testing and SARIMA Model Selection

This section contains the R code for stationarity assessment of the monthly NDVI series and SARIMA model fitting. These correspond to Sections 4.3.6 through 4.3.8 of Chapter 4.

```
# ── STATIONARITY ON MONTHLY SERIES ──────────────────────────────
adf.test(monthly_ts)    # p = 0.01 — Stationary
kpss.test(monthly_ts)   # p = 0.01 — Non-stationary (null rejected)

# ── SEASONAL AND NON-SEASONAL DIFFERENCING ──────────────────────
monthly_sdiff <- diff(monthly_ts, lag = 12)
monthly_diff  <- diff(monthly_sdiff, differences = 1)
adf.test(monthly_diff);  kpss.test(monthly_diff)  # Both confirm
stationary

# ── MANUAL SARIMA: SARIMA(1,1,1)(1,1,1)[12] ─────────────────────
manual_sarima <- Arima(monthly_ts,
  order    = c(1,1,1),
  seasonal = list(order=c(1,1,1), period=12))
summary(manual_sarima)  # AIC=-966.13 | BIC=-948.04 | RMSE=0.03896
checkresiduals(manual_sarima)  # Ljung-Box p=0.163

# ── AUTO SARIMA: exhaustive search ──────────────────────────────
auto_sarima <- auto.arima(monthly_ts,
  seasonal      = TRUE,
  stepwise      = FALSE,
  approximation = FALSE,
  trace         = FALSE)
summary(auto_sarima)    # ARIMA(2,0,0)(0,1,1)[12] with drift
                        # AIC=-990.83 | BIC=-972.72 | RMSE=0.03799
checkresiduals(auto_sarima)  # Q*(21)=12.79 | p=0.916
```

A.8 Monthly NDVI Forecast and Validation (2025)

This section provides the R code for generating the 12-month NDVI forecast for 2025 and validating it against observed values, including Figure 4.10, 4.11 and 4.12. This corresponds to Section 4.3.9 of Chapter 4.

```
# ── GENERATE 12-MONTH FORECAST ──────────────────────────────────
fc_monthly <- forecast(auto_sarima, h=12, level=95)
print(fc_monthly)

fc_df <- data.frame(
  Month     = 1:12,
  Month_Name= c('Jan','Feb','Mar','Apr','May','Jun',
                'Jul','Aug','Sep','Oct','Nov','Dec'),
  Forecast  = round(as.numeric(fc_monthly$mean), 4),
  Lo95      = round(as.numeric(fc_monthly$lower), 4),
  Hi95      = round(as.numeric(fc_monthly$upper), 4))

# ── ADD OBSERVED 2025 VALUES ────────────────────────────────────
fc_df$Actual    <- round(validation_2025$mean_ndvi, 4)
```

```
fc_df$Abs_Error <- round(abs(fc_df$Actual - fc_df$Forecast), 4)
fc_df$Pct_Error <- round((fc_df$Abs_Error / fc_df$Actual) * 100, 2)
fc_df$Residual  <- round(fc_df$Actual - fc_df$Forecast, 4)

# ── OVERALL ACCURACY METRICS ──────────────────────────────
mape_m <- mean(fc_df$Pct_Error)
mae_m  <- mean(fc_df$Abs_Error)
rmse_m <- sqrt(mean(fc_df$Residual^2))
cat('MAPE:', round(mape_m,2), '%\n')  # 9.92%
cat('MAE: ', round(mae_m,4), '\n')    # 0.0523
cat('RMSE:', round(rmse_m,4),'\n')    # 0.0644

# ── FIGURE 4.10: FORECAST VS ACTUAL LINE CHART ────────────
fc_long <- fc_df %>%
  select(Month_Name, Forecast, Actual) %>%
  tidyr::pivot_longer(cols=c(Forecast,Actual),
                      names_to='Type', values_to='NDVI')
fc_long$Month_Name <- factor(fc_long$Month_Name,
  levels=c('Jan','Feb','Mar','Apr','May','Jun',
           'Jul','Aug','Sep','Oct','Nov','Dec'))
fig_4_10 <- ggplot(fc_long, aes(x=Month_Name, y=NDVI,
              colour=Type, group=Type)) +
  geom_line(linewidth=0.9) + geom_point(size=2.5) +

scale_colour_manual(values=c('Actual'='#1a7a4a','Forecast'='#2e75b6'))
+
  labs(title='Monthly NDVI Forecast vs Actual — Aligarh District
(2025)',
       x='Month', y='Mean NDVI', colour=NULL) +
  theme_minimal(base_size=11)
print(fig_4_10)

# ── FIGURE 4.11: SCATTER PLOT ─────────────────────────────
fig_4_11 <- ggplot(fc_df, aes(x=Forecast, y=Actual)) +
  geom_point(size=3, colour='#2e75b6') +
  geom_abline(slope=1, intercept=0,
              linetype='dashed', colour='grey50') +
  labs(title='Scatter Plot: Actual vs Forecasted Monthly NDVI (2025)',
       x='Forecasted NDVI', y='Actual NDVI') +
  theme_minimal(base_size=11)
print(fig_4_11)

# ── FIGURE 4.12: RESIDUALS PLOT ───────────────────────────
fig_4_12 <- ggplot(fc_df, aes(x=factor(Month_Name,
              levels=c('Jan','Feb','Mar','Apr','May','Jun',
              'Jul','Aug','Sep','Oct','Nov','Dec')),
              y=Residual)) +
  geom_bar(stat='identity', fill='#2e75b6', colour='white') +
  geom_hline(yintercept=0, linetype='dashed', colour='grey40') +
  labs(title='Monthly Forecast Residuals — Aligarh District (2025)',
       subtitle='Positive = underestimated | Negative =
overestimated',
       x='Month', y='Residual (Actual - Forecast)') +
```

```
  theme_minimal(base_size=11)
print(fig_4_12)
```

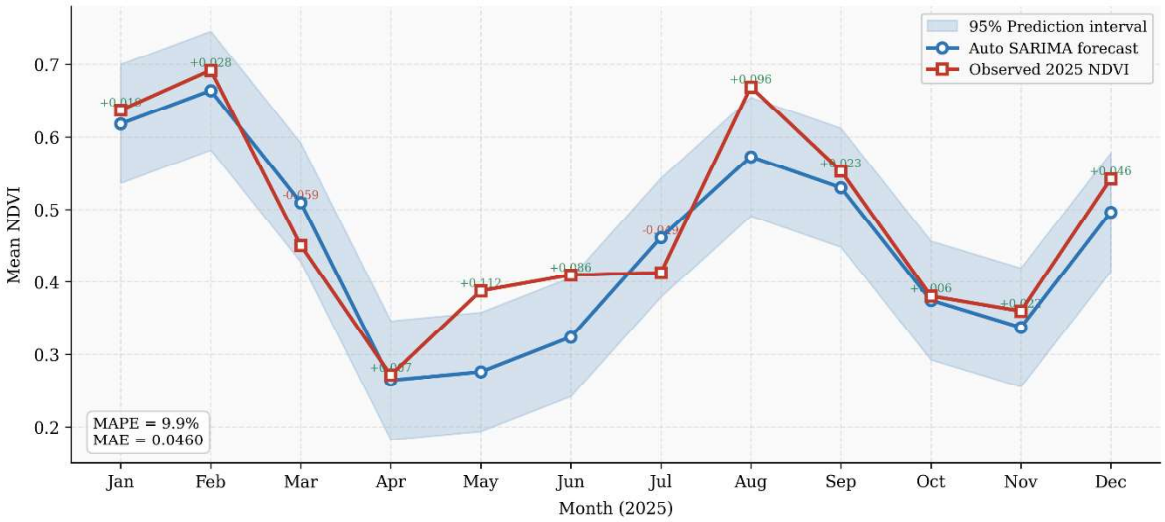


Figure 4.10: Monthly NDVI Forecast versus Actual Observed Values for Aligarh District (2025). Dark green = actual 2025 NDVI; blue = SARIMA forecast. Overall MAPE = 9.9%.

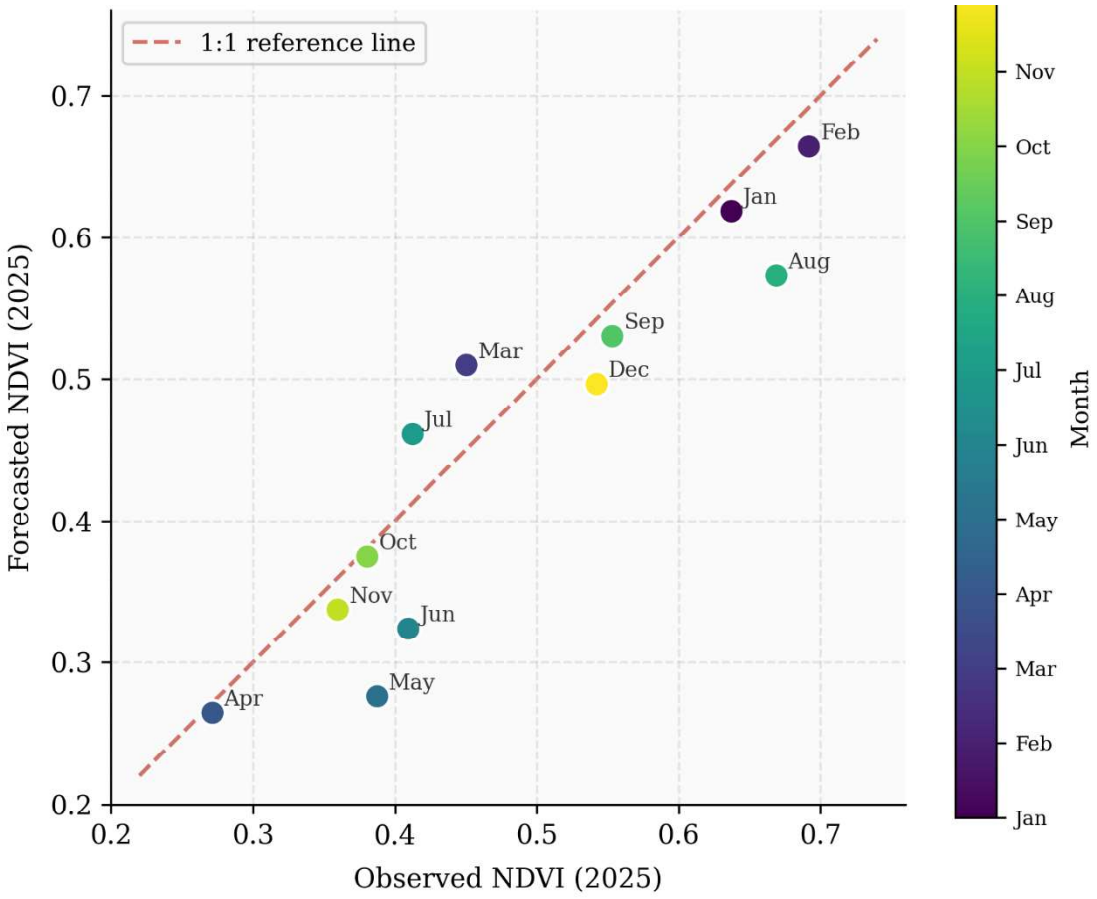
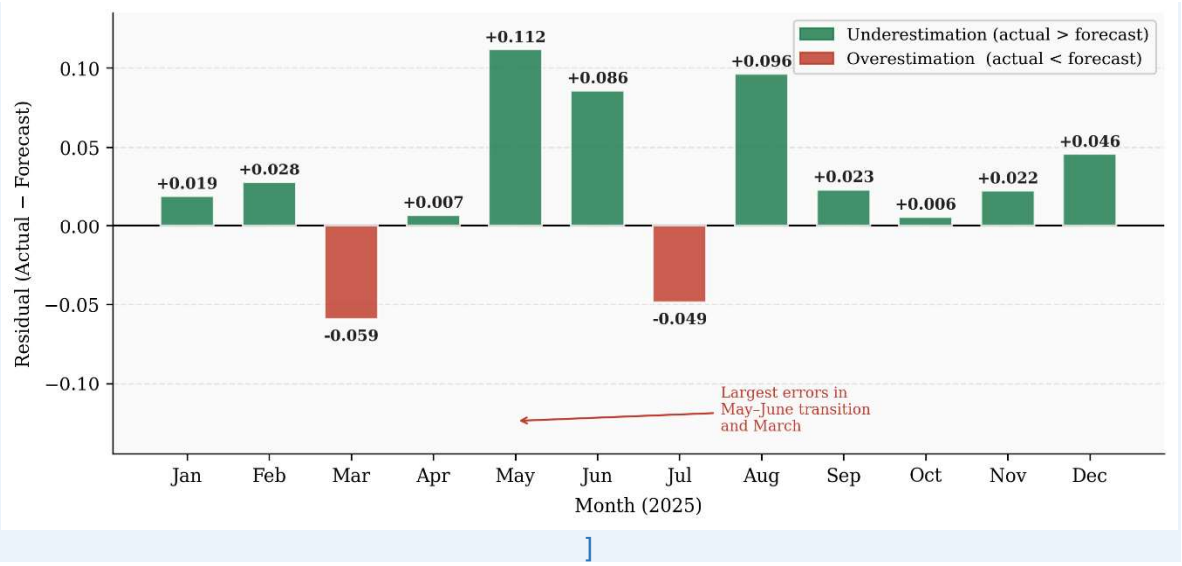


Figure 4.11: Scatter Plot of Actual versus Forecasted Monthly NDVI for Aligarh District (2025). Dashed line = perfect 1:1 agreement.



]

Figure 4.12: Monthly Forecast Residuals (Actual minus Forecast) for Aligarh District (2025). Largest errors in May–June (inter-seasonal transition period). No persistent directional bias.

A.9 Zonal Statistics Analysis

This section contains the R code for importing and preprocessing the yearly and monthly zonal statistics datasets and generating the monthly NDVI heatmap. These operations correspond to Sections 5.2 and 5.3 of Chapter 5.

A.9.1 Yearly Zonal Statistics Import

```
zonal_yearly <- read_csv('yearly_zonal_stats_2001_2024.csv')
zonal_yearly <- zonal_yearly %>%
  select(year,NDVI_max,NDVI_mean,NDVI_median,NDVI_min,NDVI_stdDev) %>%
  rename(Year=year, Max=NDVI_max, Mean=NDVI_mean,
         Median=NDVI_median, Min=NDVI_min, StdDev=NDVI_stdDev)
str(zonal_yearly); colSums(is.na(zonal_yearly))
```

A.9.2 Monthly Zonal Statistics and NDVI Heatmap (Figure 5.1)

```
zonal_monthly <- read_csv('monthly_zonal_stats_2001_2024.csv')
zonal_monthly <- zonal_monthly %>%
  select(year, month, NDVI_mean) %>%
  rename(Year=year, Month=month, Mean=NDVI_mean)

fig_5_1 <- ggplot(zonal_monthly,
  aes(x=factor(Month, labels=c('Jan','Feb','Mar','Apr','May','Jun',
                'Jul','Aug','Sep','Oct','Nov','Dec')),
      y=factor(Year), fill=Mean)) +
  geom_tile(colour='white', linewidth=0.3) +
  scale_fill_gradientn(
    colours = c('#ffffe0','#addd8e','#41ab5d','#006837'),
    name    = 'Mean NDVI') +
  labs(title='Monthly NDVI Heatmap — Aligarh District (2001–2024)',
       x='Month', y='Year') +
  theme_minimal(base_size=10) +
  theme(axis.text.y = element_text(size=7))
print(fig_5_1)
```

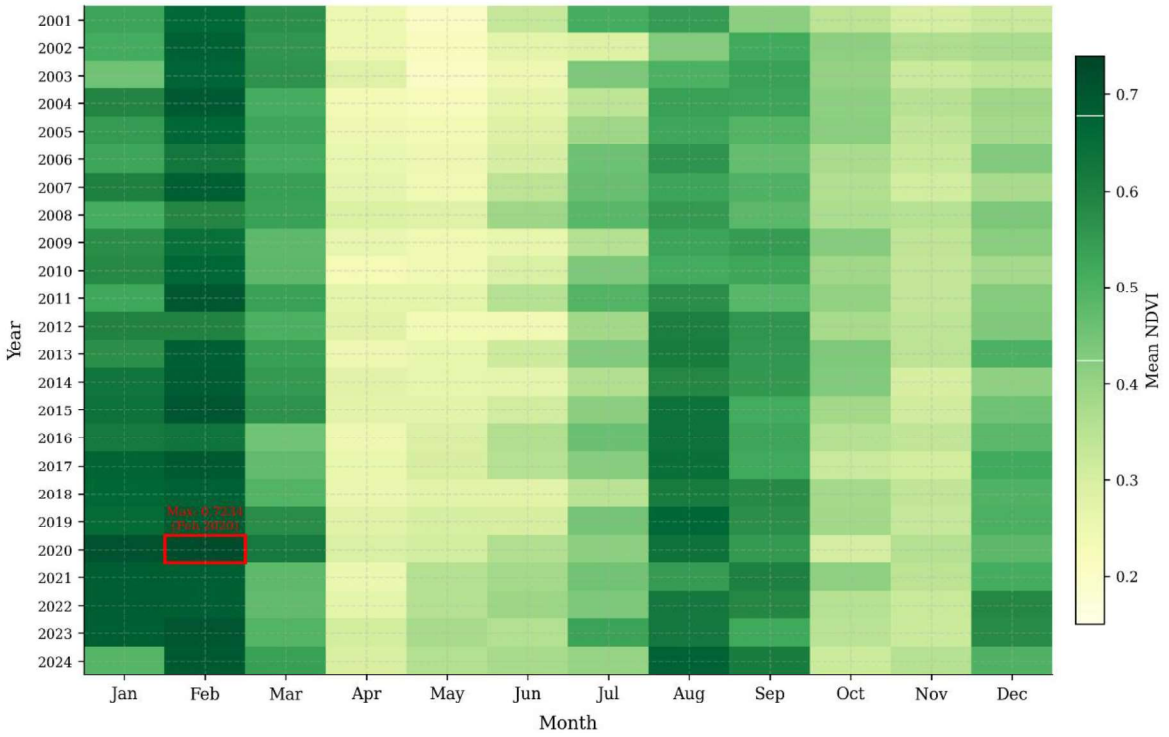


Figure 5.1: Monthly NDVI Heatmap for Aligarh District (2001–2024). Each cell = mean NDVI for a given month-year combination. Darker green = denser vegetation. February column shows consistently high values across all years (Rabi peak). April–May columns show consistently low values (inter-seasonal trough).

A.9.3 Yearly Zonal Statistics Trend Plot (Figure 5.5)

```
fig_5_5 <- ggplot(zonal_yearly, aes(x=Year)) +
  geom_ribbon(aes(ymin=Mean-StdDev, ymax=Mean+StdDev),
              fill='#c7e9c0', alpha=0.5) +
  geom_line(aes(y=Max),    colour='#2e75b6', linewidth=0.9) +
  geom_line(aes(y=Mean),   colour='#1a7a4a', linewidth=1.1) +
  geom_line(aes(y=Median), colour='darkorange', linewidth=0.8,
            linetype='dashed') +
  labs(title='Yearly Zonal Statistics — Aligarh District (2001–2024)',
       subtitle='Blue=Max | Green=Mean | Orange dashed=Median |
Band=Mean±SD',
       x='Year', y='NDVI') +
  theme_minimal(base_size=11)
print(fig_5_5)
```

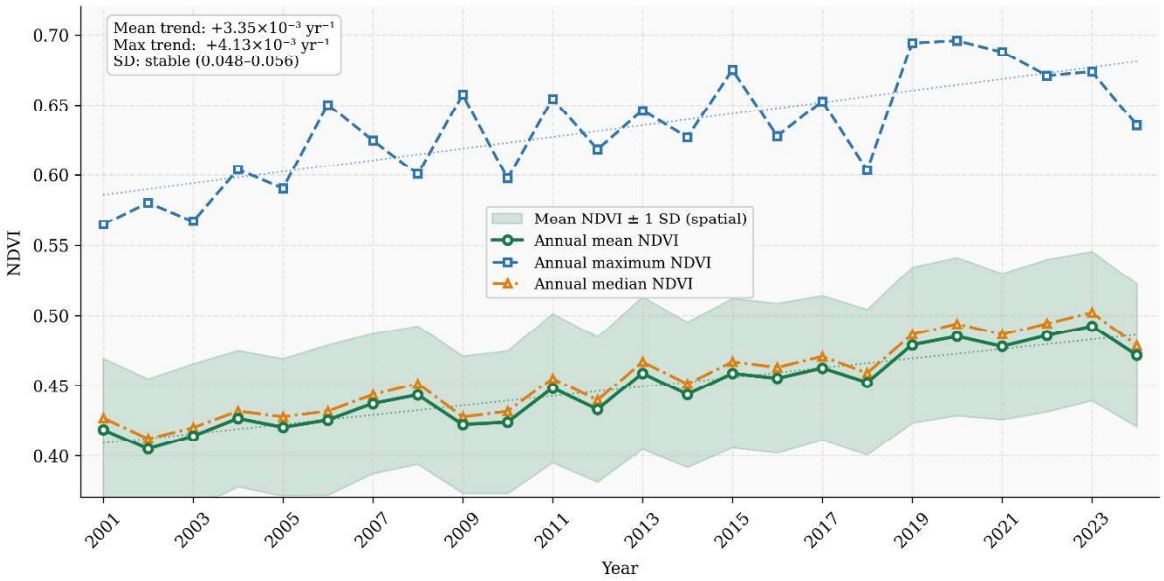


Figure 5.5: Yearly Zonal Statistics for Aligarh District (2001–2024). Green shaded band = mean ± 1 SD; blue line = annual maximum NDVI; orange dashed line = annual median NDVI. All three measures show consistent improvement. Spatial standard deviation band remains stable (0.048–0.056).

A.10 Bayesian Linear Regression Analysis

This section contains the complete R code for fitting the Bayesian linear regression model using rstanarm, extracting posterior summaries, assessing MCMC convergence, and performing the posterior predictive check. These operations correspond to Section 5.4 of Chapter 5.

A.10.1 Bayesian Model Fitting

```
bayes_model <- stan_glm(
  NDVI ~ Year,
  data             = ndvi_data,
  family           = gaussian(),
  prior            = normal(location=0, scale=2.5),
  prior_intercept  = normal(location=0, scale=2.5),
  chains           = 4,
  iter             = 4000,    # 2000 warmup + 2000 sampling per chain
  seed             = 123,     # Fixed for reproducibility
  refresh          = 0        # Suppress sampling messages
)
```

A.10.2 Posterior Summary Extraction

```
print(bayes_model, digits=6)
posterior_summary(bayes_model)
posterior_interval(bayes_model, prob=0.95)

posterior_draws <- as.data.frame(bayes_model)
prob_positive   <- mean(posterior_draws$Year > 0)
cat('P(slope > 0):', round(prob_positive,6), '\n')  # 1.000000

rhat_values <- rhat(bayes_model)      # All = 1.00
neff_ratio(bayes_model)               # All > 0.5
```

A.10.3 MCMC Trace Plots (Figure 5.2)

```
trace_plot <- mcmc_trace(as.array(bayes_model),
  pars = c('Year','(Intercept)')) +
  labs(title='MCMC Trace Plots — Bayesian NDVI Trend Model',
       subtitle='4 chains, 2,000 post-warmup iterations | Rhat =
1.00')
print(trace_plot)
```

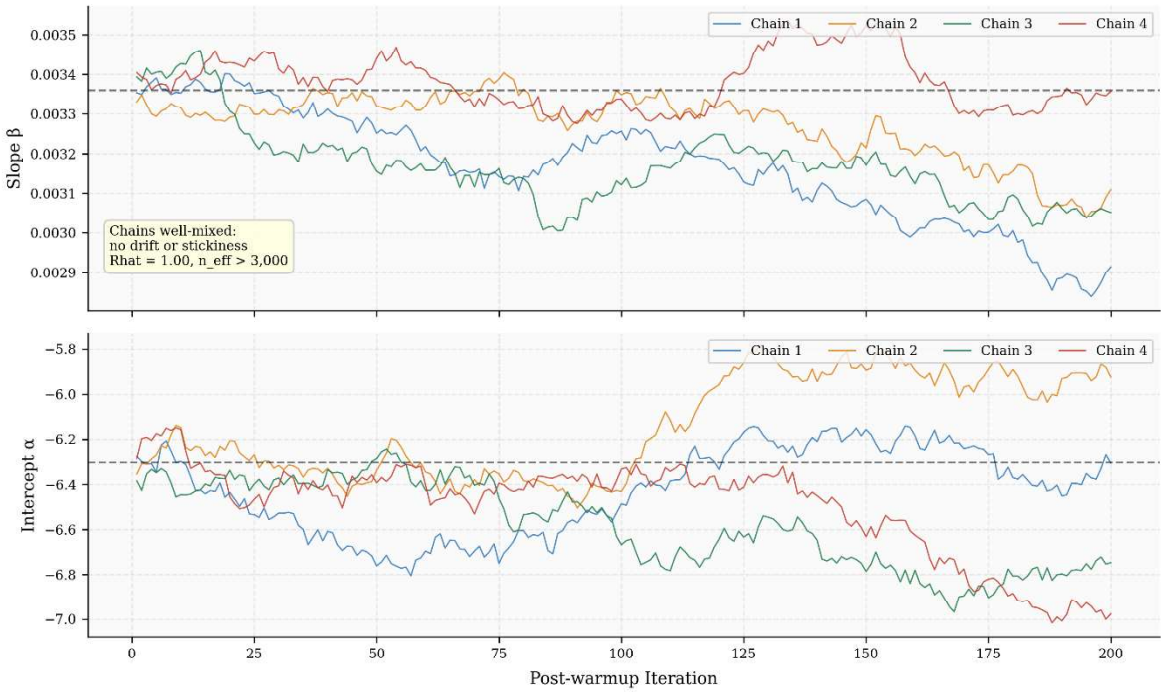


Figure 5.2: MCMC Trace Plots for Bayesian Regression Parameters (4 chains, 2,000 post-warmup iterations each). All chains well-mixed with no drift. Rhat = 1.00 for all parameters; n_eff > 3,000.

A.10.4 Posterior Density Plot (Figure 5.3)

```
density_plot <- mcmc_areas(as.array(bayes_model),
  pars = 'Year', prob = 0.95) +
  labs(title='Posterior Density — Annual NDVI Slope Parameter (β)',
       subtitle='Shaded region = 95% Credible Interval |
OLS=0.003353',
       x='Slope β (NDVI units per year)')
```

```
print(density_plot)
```

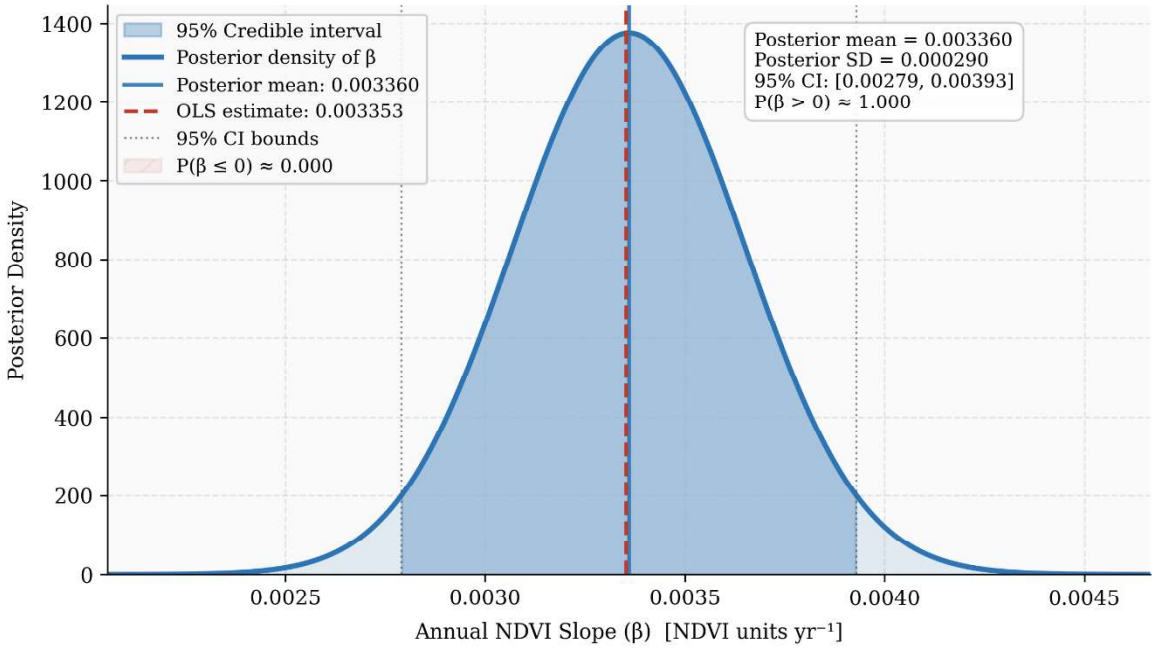


Figure 5.3: Posterior Probability Distribution of the Annual NDVI Slope Parameter (β). Distribution centred at 0.003360 (SD = 0.000290). Shaded region = 95% credible interval [0.00279, 0.00393]. Entire distribution is positive: P(β > 0) ≈ 1.000.

A.10.5 Posterior Predictive Check (Figure 5.4)

```
pp_plot <- pp_check(bayes_model, nreps=50) +
  labs(title='Posterior Predictive Check — Bayesian NDVI Model',
       subtitle='Blue = 50 replicated datasets | Green = observed
NDVI',
       x='Annual Mean NDVI')
print(pp_plot)
```

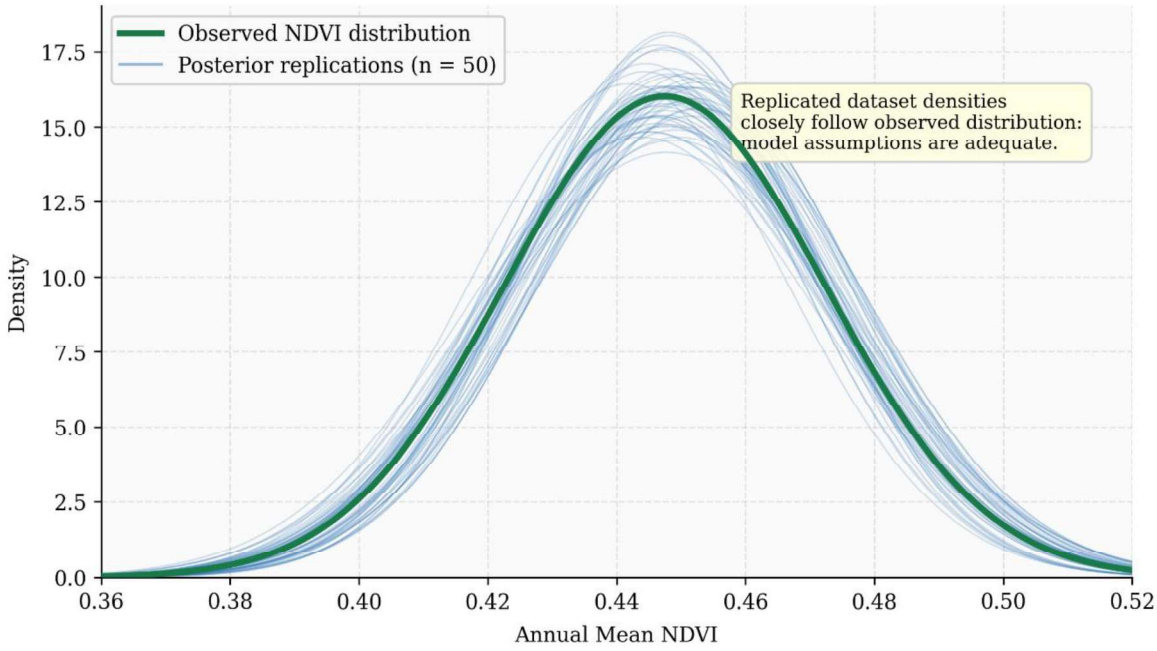


Figure 5.4: Posterior Predictive Check for the Bayesian Linear Regression Model. Blue curves = density of 50 synthetic NDVI datasets drawn from the posterior; green curve = observed NDVI density. Close agreement confirms model adequacy.

A.11 Figure Export Code

This section provides the R code for exporting all dissertation figures at 300 dpi as PNG files for insertion into the Word document. All figures are exported to the working directory.

```
# ── CHAPTER 4 FIGURES ──────────────────────────────────────

ggsave('Figure4_1_Annual_NDVI_Trend.png',
       plot=fig_4_1, width=9, height=5, dpi=300, units='in')

png('Figure4_2_Regression_Diagnostics.png',
    width=9, height=7, units='in', res=300)
par(mfrow=c(2,2)); plot(trend_model); dev.off()

ggsave('Figure4_3_Sens_Slope.png',
       plot=fig_4_3, width=9, height=5, dpi=300, units='in')

png('Figure4_4_ACF_PACF.png', width=10, height=4.5, units='in',
res=300)
par(mfrow=c(1,2))
acf(ndvi_diff, lag.max=12); pacf(ndvi_diff, lag.max=12)
dev.off()
```

```
ggsave('Figure4_5_ARIMA_Forecast.png',
       plot=fig_4_5, width=10, height=5, dpi=300, units='in')
ggsave('Figure4_6_Validation_Bar.png',
       plot=fig_4_6, width=7, height=5, dpi=300, units='in')
ggsave('Figure4_7_Monthly_TimeSeries.png',
       plot=fig_4_7, width=11, height=4.5, dpi=300, units='in')
ggsave('Figure4_8_Monthly_Seasonal_Avg.png',
       plot=fig_4_8, width=9, height=5.5, dpi=300, units='in')

png('Figure4_9_STL_Decomposition.png',
    width=10, height=7, units='in', res=300)
plot(stl_result,
     main='STL Decomposition of Monthly NDVI — Aligarh District')
dev.off()

ggsave('Figure4_10_Monthly_Forecast_vs_Actual.png',
       plot=fig_4_10, width=9, height=5, dpi=300, units='in')
ggsave('Figure4_11_Scatter_Validation.png',
       plot=fig_4_11, width=7, height=6, dpi=300, units='in')
ggsave('Figure4_12_Monthly_Residuals.png',
       plot=fig_4_12, width=9, height=5, dpi=300, units='in')

# ── CHAPTER 5 FIGURES ──────────────────────────────────────
ggsave('Figure5_1_Monthly_Heatmap.png',
       plot=fig_5_1, width=11, height=7, dpi=300, units='in')
ggsave('Figure5_2_MCMC_Trace.png',
       plot=trace_plot, width=10, height=6.5, dpi=300, units='in')
ggsave('Figure5_3_Posterior_Density.png',
       plot=density_plot, width=8, height=5, dpi=300, units='in')
ggsave('Figure5_4_Posterior_Predictive.png',
       plot=pp_plot, width=8, height=5, dpi=300, units='in')
ggsave('Figure5_5_Zonal_Statistics.png',
       plot=fig_5_5, width=10, height=5.5, dpi=300, units='in')
```